

Essay 1a: What is Ultimate Tic-Tac-Toe?

This is the first in a series of essays exploring how an AlphaZero-style AI learns to play Ultimate Tic-Tac-Toe. We start with the game itself — not because the rules are complicated, but because understanding why the game is hard sets up everything that follows.

The Game You Already Know

You already know Tic-Tac-Toe. Two players, a 3×3 grid, first to three in a row wins. The game is so simple that a child can master it in an afternoon — and once mastered, it stops being interesting. Every game between two competent players ends in a draw. There is nothing left to discover.

Ultimate Tic-Tac-Toe fixes this by nesting the game inside itself.

The Rules

The board is a 3×3 grid of smaller 3×3 grids — nine local boards arranged in a larger global board. Each local board is a standard game of Tic-Tac-Toe. The global board tracks which player has won each local board.

The twist is in how moves work. When you place your mark in a square of a local board, the position of that square within its local board determines which local board your opponent must play in next. Put your mark in the top-right square of any local board, and your opponent's next move must be made somewhere in the top-right local board.

If your opponent is sent to a local board that is already decided (won or drawn), they may play in any open local board of their choosing.

The game ends when one player wins three local boards in a row on the global board — horizontal, vertical, or diagonal.

Why This Is Hard

Regular Tic-Tac-Toe has nine squares. From the first move, there are nine choices; from the second, eight; and so on. The total number of possible games is at most $9! = 362,880$ — small enough to enumerate by hand.

Ultimate Tic-Tac-Toe is different. Each local board has nine squares, and at any point in the game you may have access to any one of them. On the very first move, there are nine choices of local board and nine choices of square within it, giving eighty-one possibilities. After that, the “send your opponent” rule constrains the active local board — but within that board, all available squares are legal. In practice, each position has roughly nine legal moves on average, and games last on the order of forty moves.

That gives a game tree of roughly 9^{40} — about 10^{38} positions. To put that in perspective: there are estimated to be around 10^{80} atoms in the observable universe. The game tree of Ultimate Tic-Tac-Toe is larger than the number of atoms on Earth, and smaller than the number of atoms in the galaxy. It occupies a strange middle ground: too large for exhaustive search, small enough that patterns exist.

There is a related but distinct quantity worth distinguishing. The *game tree* counts sequences of moves — paths through the space of possible play. But many different sequences can lead to the same board *configuration*. The number of distinct board states is bounded by 3^{81} (three possibilities — empty, X, or O — for each of eighty-one squares), which is roughly 4.4×10^{38} . In practice, most of these states are unreachable given the game’s rules, but the point stands: the *state space* and the *game tree* are different objects. We will need to be precise about which one we mean as the series progresses.

Local Decisions, Global Consequences

What makes Ultimate Tic-Tac-Toe genuinely strategic — and not just tactically complex — is the tension between local and global objectives.

This interaction between levels is what makes the game interesting for AI research. The right move in any position depends not just on the current local board but on the global pattern, the opponent’s options, and the cascading constraints several moves ahead. It is a game where thinking locally is a guaranteed path to defeat.

What It Takes to Play Well

A strong player of Ultimate Tic-Tac-Toe needs several things simultaneously: the ability to evaluate any given board position, the ability to look ahead through a branching tree of possibilities, and some principled way to balance deep search against broad survey.

Humans develop these abilities through experience — playing many games, losing in-

structively, gradually forming intuitions about which local boards matter and which positions are tactically lost. We don't enumerate 9^{40} positions. We build compressed, imperfect models of the game and act on them.

The question, then, is whether a machine can do the same. Not by brute force enumeration — the tree is too large for that — but by *learning from experience*. By playing games, observing outcomes, and updating some internal representation of what positions are promising and what moves tend to lead where.

That is the question the rest of this series is about.

Next: Essay 1b introduces reinforcement learning — the framework that lets an agent improve by interacting with an environment. We will see how the problem of learning from games turns out to have a clean mathematical structure, and why that structure is harder to exploit than it first appears.

Code: [Notebook 1 — The UTTT Game Engine](#) implements everything described here — board representation, the send-your-opponent rule, win detection, and a random-agent tournament — in runnable Python using the production game engine.